

File Structure

1 Basic Idea

Definition

File structure describes how the contents of a file are organized and what meaning the operating system gives to that organization.

- A file can be viewed in several different ways.
- The file structure determines what the operating system understands about the file.
- Some systems treat files as raw byte sequences.
- Other systems treat files as records or indexed trees.
- The more structure the OS understands, the more help it can provide.
- The less structure the OS enforces, the more flexible files become for user programs.

2 Three Common File Structures

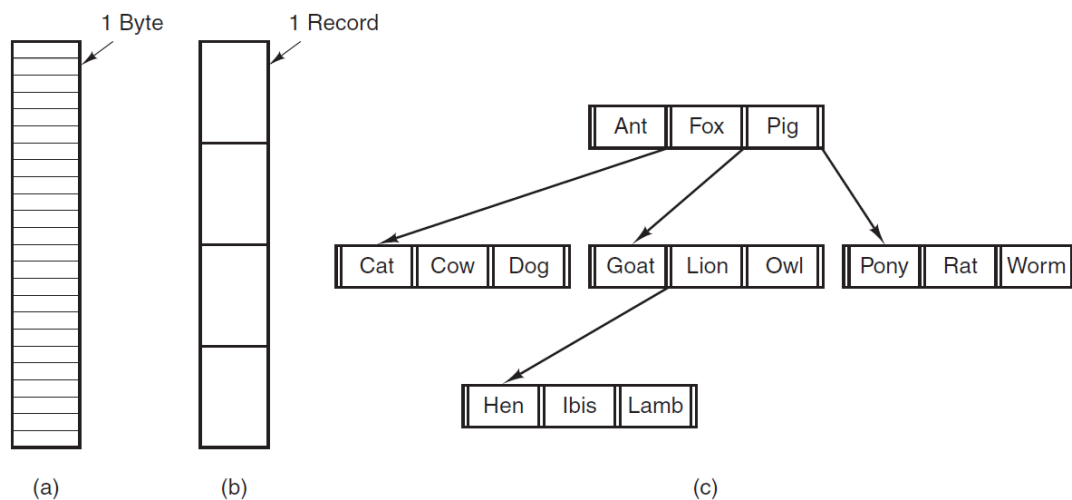


Figure 4-2. Three kinds of files. (a) Byte sequence. (b) Record sequence. (c) Tree.

Structure	Main idea
Byte sequence	The file is an unstructured sequence of bytes.
Record sequence	The file is a sequence of fixed-length records.
Tree	The file is a tree of records sorted by a key field.

3 Byte Sequence Files

Byte Sequence

A **byte sequence file** is a file whose contents are treated by the operating system as a simple sequence of bytes.

- The operating system does not know what the bytes mean.
- The operating system does not enforce records, fields, or keys.
- Any meaning is imposed by user-level programs.
- A text editor may interpret bytes as characters.
- A compiler may interpret bytes as source code.
- An image viewer may interpret bytes as encoded image data.

Used By

UNIX, Linux, OS X, and Windows use the byte-sequence file model.

4 Why Byte Sequences Are Flexible

- Programs can store any format they want.
- The OS does not need to understand every possible file type.
- New application file formats can be created without changing the OS.
- The OS does not help interpret file contents.
- The OS also does not get in the way of unusual file formats.

Trade-Off

The byte-sequence model gives maximum flexibility, but the responsibility for interpreting the file belongs to user programs.

5 Record Sequence Files

Record Sequence

A **record sequence file** is a file made of fixed-length records, where each record has some internal structure.

- The file is read and written in record units.
- A read operation returns one record.

- A write operation overwrites or appends one record.
- Records are usually the same size.
- This model was common on older mainframe systems.

6 Historical Record Examples

- Older systems often used 80-character records.
- This matched 80-column punched cards.
- Some systems also supported 132-character records.
- This matched old 132-column line printers.
- Programs read input in 80-character units.
- Programs wrote output in 132-character units.
- Current general-purpose systems rarely use this as their main file model.

Historical Note

Record-sequence files matched the physical media and printers used by older mainframe systems.

7 Tree-Structured Files

Tree File

A **tree-structured file** stores records in a tree, usually sorted by a key field in each record.

- Records do not have to be the same length.
- Each record contains a key field.
- The key field is located at a fixed position in each record.
- The tree is sorted by the key.
- This allows fast searching by key.
- The main operation is often lookup by key, not reading the next record.

Example

In a tree-structured zoo file, the system could directly search for the record with key **pony** without the user knowing its physical position.

8 Adding Records in Tree Files

- New records can be added by key.
- The operating system decides where the new record should be placed.
- The user does not manually choose the physical position.
- This is very different from the byte-sequence model.
- Tree-structured files have been used on large mainframe systems for commercial data processing.

9 Comparison

File structure	Strength	Weakness
Byte sequence	Maximum flexibility; programs can define any format.	OS gives no help interpreting file contents.
Record sequence	Simple record-based read and write operations.	Inflexible for data that does not naturally fit fixed-size records.
Tree	Fast lookup by key and OS-managed placement.	More complex and less general than byte streams.

10 Key Contrast

- Byte-sequence files are application-defined.
- Record-sequence files are operation-defined around fixed records.
- Tree-structured files are search-defined around keys.
- Modern general-purpose systems prefer byte sequences because they are simple and flexible.
- Specialized systems may use records or trees when the OS needs to support structured access directly.

Final Point

The more structure the operating system imposes on files, the more services it can provide, but the less freedom applications have to define their own formats.